

A developer who digs into root causes and enjoys solving difficult problems

AI Application Developer · Backend Engineer

kkuldangi2@gmail.com GitHub ↗ Blog ↗ LinkedIn ↗

SUMMARY

RAG evidence selection: coverage 66.25% → 91.25%, false evidence 20 → 0

Agent execution environment: user-defined skills behind approval, edit quality 94/114 → 104/114

Data pipelines: PDF conversion pass rate 45.17% → 89.89%, drug conversion \$200 estimated → \$11.18 spent

PROJECTS & EXPERIENCE

Fruition

2026.04 — Present

LLM Wiki-based AI workspace · AI SW Maestro, 17th cohort · Deployed on AWS EKS, in QA

Team project (3 + designer) / AI lead

How it started: a teammate's pain point with managing documents, refined through design thinking — saved documents could not be found when needed.

Team: me (all AI), 1 backend (Spring backend), 1 frontend & DevOps (frontend, MSA deployment on AWS), external designer · 3 mentors

GitHub · AI repository ↗

Improving retrieval for grounded AI answers

Retrieved document evidence for AI answers. Coverage on 80 answerable questions rose from 66.25% to 91.25%, while false evidence returns on 20 unanswerable questions fell from 20 to 0.

- Evidence selection for RAG

Problem: Retrieval missed required facts and returned evidence for unanswerable questions.

Solution: Applied Jev after routing tests, then controlled candidates and return limits after a full-corpus trial. Tuned to 300 candidates and up to 4 parallel batches.

Result: Across 9 papers, coverage on 80 answerable questions rose from 66.25% to 91.25%. False evidence returns on 20 unanswerable questions fell from 20 to 0. Median time before/after Jev tuning: 9.090s → 2.251s.

Reusable Skills and improved AI Agent routing

Built reusable Skills with user approval before changes. Applied Jev to route requests to questions, edits and other tasks: correct decisions rose from 77/98 to 81/98, and median decision time fell from 8.967s to 0.658s.

- User-defined task instructions (Skills)

Problem: Developers cannot build every recurring workflow users need.

Solution: Built drafting, review and publishing of natural-language instructions, with checks against bypassing permissions or approval. Document changes require separate preview and approval.

Result: Published Skills became available to the Agent. In a 6-document check, no changes occurred before approval; 3 folders were created and documents moved afterward.

- Jev-based request routing

Problem: Misclassifying questions as edits led to the wrong action.

Solution: Applied Jev, a model that picks only from a fixed set of options, to select task types and additional conditions for questions, edits, document creation and folder organization.

Result: Fully correct decisions on the same 98 requests rose from 77 to 81 (63 for GPT-5-nano on the same options). Median decision time fell from 8.967s to 0.658s.

Conversation context and edit input

Holding context across a long conversation while shrinking what the model receives for an edit.

- Conversation memory

Problem: a long conversation cannot be passed to the model in full.

Approach: keep the last six turns verbatim, compress older ones into a rolling summary, and keep the previous summary if summarization fails. The router also sees earlier routing decisions.

Result: referring expressions resolve against earlier turns and feed into routing.

- Bounding the edit input

Problem: sending a whole document drives up cost and latency.

Approach: send only the target line range plus context, and reject ranges straddling a table or code-block boundary. A benchmark tracks input size against the source, plus average and p95 time.

Result: edit input stays centred on the target range at any document length.

Document conversion pipeline and faster Wiki generation

Converted uploaded PDFs for retrieval and grounded answers, and reduced Wiki generation time. Internal pass rate on 30 pages improved from 45.17% to 89.89%;

analysis of 4 documents fell from 282.11s to 73.89s.

- PDF → Markdown conversion
Problem: Open-source conversion damaged tables and equations; full AI conversion still had errors and high API usage.
Solution: Compared PDF tools and adopted AnyDoc. Repaired damaged text with AI, processed tables and equations separately, and preserved original figures.
Result: Internal pass rate on 30 PDF pages improved from 45.17% to 89.89%.
- Parallel Wiki generation
Problem: Later documents waited for earlier external AI responses.
Solution: Compared worker counts and requests per worker. Capped workers at 4 to manage memory; rejected extra concurrency and GPU parallelism that slowed processing.
Result: Analysis of 4 documents fell from 282.11s to 73.89s (about 74% less).

Service responsibilities, communication and LLM tracing

Defined responsibilities, data ownership and communication for services including AI, and added tracing that exposes each stage of an AI run.

- Proposed the initial MSA design, then jointly finalized service boundaries, data ownership and communication with teammates and mentors.
- LLM run tracing and the boundary around sensitive data
Problem: the evaluation loop ran as plain Python, so extraction, evaluation and retry could not be inspected separately — yet tracing everything would send raw tool lookups and personal data outside.
Approach: rebuilt the loop as a LangGraph graph so generation, evaluation and retry each became a traced node. A missing LangSmith key skips tracing but still runs the graph. The agent graph keeps tracing off, excludes raw tool lookups from persisted state, and deletes that state ahead of the run record at end of retention. My own prompt log masks numeric personal data before writing.
Result: graph nodes and model call spans are visible on real requests. Timing runs disable tracing and prompt logging so instrumentation stays out of the numbers.

Internal evaluations. RAG timing compares Jev before/after tuning. Parallel document timing covers analysis only, excluding storage and post-processing.

See the detailed portfolio for the problem-solving process and experimental results.

[RAG evidence selection ↗](#) [Agent execution environment ↗](#) [PDF conversion ↗](#) [Parallel document processing ↗](#)

Pilltip

2025.03 — 2025.12

Personalized AI medication service · Pusan National University capstone

Team project (3 + designer) / Core backend · data preparation

How it started: drug information is scattered, making it hard to check whether a medicine is safe for your current health condition.

Team: me (backend & AI), 1 Android (Kotlin app), 1 backend & security (user DB, login, questionnaires, AES-GCM), designer. The features below are my work.

[GitHub repository ↗](#)

Drug data preparation pipeline

Rewrote about 44,000 drug records in plain language. Deduplication reduced input from 1GB to 326MB; after filling missing information, conversion cost \$11.18 in actual API spending versus about \$200 estimated for the original text.

- **Problem:** Converting all 44,000 drug records was estimated to cost about \$200 in API calls.
Solution: Removed exact duplicates, then semantically equivalent sentences. Filled the missing 5% with ingredient-specific precautions, converted via GPT Batch API and mapped results back to each drug.
Result: Conversion input shrank from 1GB to 326MB. Actual API spending was \$11.18 versus about \$200 estimated for all original text.

Estimated cost versus actual spending; preprocessing labor is excluded.

AI medication chatbot using health profiles

Found drug and medication information from natural-language symptoms. Evaluated health profiles internally, separately from information sent to external AI.

- **Problem:** Everyday symptom language differed from drug indications, while sensitive health data needed to stay out of external AI inputs.
Solution: Used Weaviate semantic search and AI orchestration for drug discovery, medication risks and dosage. Evaluated pregnancy, current medications and underlying conditions in internal code.
Result: Implemented personalized symptom-based search while excluding those profile fields from external LLM inputs.

Backend feature implementation

Built server features for drug search, risk information, medication reminders and logs, and family profiles. Integrated with teammates responsible for the mobile app, users and security.

- Designed the drug database on Java and Spring Boot with search, autocomplete and DUR risk display.
- Built FCM medication reminders and logs, deep-link friend invites and family profile switching, and kept API docs, meeting notes and ADRs in Notion to share decisions.

Feature work driven by feedback from live exhibitions

We ran the booth as Pusan National University's representative exhibition team, and I fed the feedback we collected into the next slice of work. A request from the first exhibition became a feature within the data we already had.

- Extending support to health functional foods
Feedback: at the first exhibition, visitors asked to check health functional foods alongside prescription drugs.
What shipped: within the health functional food data we held, I added them to search and autocomplete, surfaced caution and co-administration information, allowed them in medication reminders and logs, and included them in chatbot answers.
Scope: limited to the data we had obtained; not the exhaustive coverage we have for drugs.
- Feedback I could not act on
Someone pointed out that the service would be better off running in containers, but I could not adopt it inside the capstone timeline. On Fruition I designed the AI services around a container-based deployment from the start.

See the detailed portfolio for the problem-solving process and experimental results.

[Data pipeline case study ↗](#) [Personalization case study ↗](#)

Skills

AI & Search

LangChain · LangGraph · Spring AI · RAG · LLM Evaluation · Prompt injection defense
LangSmith · Elasticsearch · Weaviate

Backend

Java · Spring Boot · Python

Data & Infra

MySQL · PostgreSQL · Redis · Kafka
Docker · Docker Compose · GitHub Actions

Activities & contributions

2026.06 — 2026.06

Rhwp open-source contributions

Investigated bugs, submitted fixes and addressed CI in a Rust HWP/HWPX project. Fixed textFlow attributes of images, tables and shapes being reset after saving.

[GitHub repository ↗](#) [PR #1213 · Merged ↗](#) [PR #1351 · Submitted ↗](#)

2025.03 — 2026.01



APPTIVE, Pusan National University

Backend mentee and mentor. Led 6 mentoring and code-review sessions for 12 mentees. Helped revise the curriculum to strengthen HTTP, Servlet, REST API and database foundations.

APPTIVE Backend Mentor Merit Award · 2026.01.09

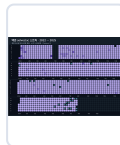
2025.11 / 2025.07



SK AI SUMMIT · K-ICT WEEK in Busan

Represented Pusan National University at the Pilltip booth, demonstrating the service and answering technical questions.

2022.02 — 2025.07



945-day Baekjoon problem-solving streak

Aiming for one problem a day, I reached a longest streak of 945 consecutive days on solved.ac. 1,659 problems solved, solved.ac Platinum IV. Solutions are recorded in a GitHub repository.

[Baekjoon solutions repository ↗](#)

Education & certification

2022.02 — 2026.02
(Transfer)

Pusan National University

GPA 4.12 / 4.5

2019.03 — 2022.02
(Withdraw)

Daegu University

GPA 4.2 / 4.5

2025.09

Engineer Information Processing

Human Resources Development Service of Korea

Awards

2025.10



Capstone Design Gold Award

Software & AI division / College of Information and Biomedical Engineering, Pusan National University

2025.09



Busan DATA WEEK Top Excellence Award
Data utilization case competition / Busan Techno Park

2025.09



AI LAUNCH Career School Startup Hackathon Encouragement Award
KRYPTON X x Root Impact x Google.org

2025.08



SW University Digital Competition Sponsor Award
SW University Council